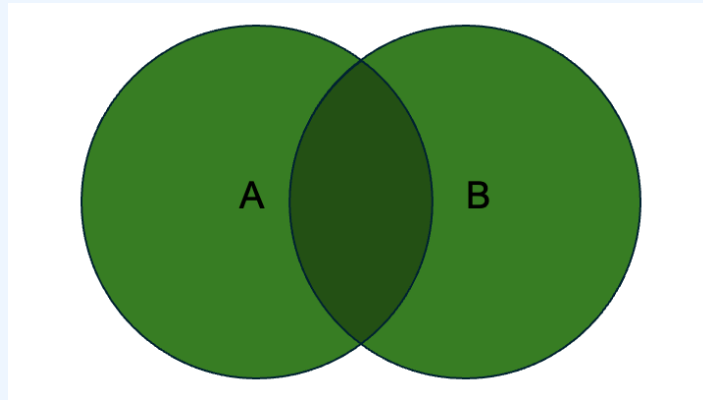


B 4.1.5 Sets as an ADT Python (HL Only)

B4.1.5: A Python Guide to Sets

In computer science, we often need to work with groups of items. You are already familiar with lists, which are ordered collections. But what if you need a collection where the order doesn't matter, and you want to ensure that every item is unique? For this, we use a powerful Abstract Data Type (ADT) called a **Set**.

This chapter will guide you through the fundamental characteristics of sets, the core operations you can perform on them, and how to use them to solve logical problems in Python.



1. The Fundamental Characteristics of Sets

A set is a collection that follows two simple but strict rules:

1. **Unique Elements:** A set cannot contain duplicate items. If you try to add an element that is already in the set, the set simply remains unchanged. This is perfect for when you need to keep track of unique items, like the usernames of all students registered for an exam.
2. **Unordered:** The elements in a set have no defined order or index. You cannot ask for "the second element" of a set because there is no concept of a first, second, or last element. Think of it like a bag of marbles—you know which marbles are in the bag, but they have no fixed position inside it.

Because of these properties, sets are extremely fast for checking if an element is present in the collection, which is one of their main advantages.

Creating a Set in Python

You can create a set from a list (which automatically removes duplicates) or by using curly braces {}.

```
# From a list
my_list = [1, 2, 2, 3]
my_unique_items = set(my_list) # Becomes {1, 2, 3}

# Using curly braces
my_set = {10, 20, 30}

# IMPORTANT: To create an empty set, you MUST use set()
empty_set = set()
```

2. Core Operations on a Single Set

These are the fundamental actions you'll need to manipulate the contents of a set.

Adding an Element

Use the `.add()` method.

```
my_set = {10, 20}
my_set.add(30) # my_set is now {10, 20, 30}
my_set.add(10) # my_set is still {10, 20, 30}
```

Checking if an Element is in a Set

Use the `in` keyword.

```
my_set = {10, 20, 30}
print(20 in my_set) # True
print(99 in my_set) # False
```

Removing an Element

You have two options, and it's important to know the difference:

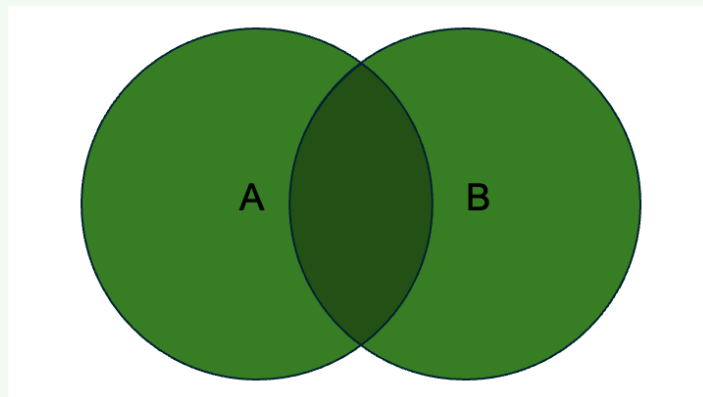
- `.remove(element)`: Removes the element. If the element is not found, it will cause a **KeyError** and crash your program. Use this when you are sure the item exists.
- `.discard(element)`: Removes the element. If the element is not found, it does **nothing**. This is the safer option if you are not sure the item exists.

3. Operations Between Multiple Sets

This is where the power of sets really shines, allowing you to compare and combine collections of data logically. We can visualize these operations using **Venn Diagrams**.

Union: Combining Sets

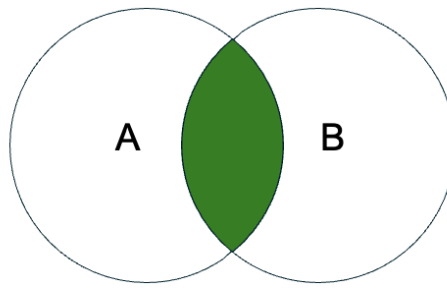
The **union** of two sets creates a new set containing all the unique elements from *both* sets. Use the pipe operator `|` or the `.union()` method.



```
A = {1, 2, 3}
B = {3, 4, 5}
C = A | B # C is {1, 2, 3, 4, 5}
```

Intersection: Finding Common Elements

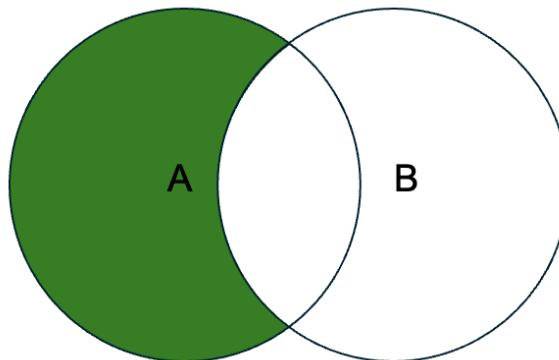
The **intersection** of two sets creates a new set containing only the elements that exist in *both* original sets. Use the ampersand operator `&` or the `.intersection()` method.



```
A = {1, 2, 3}
B = {3, 4, 5}
C = A & B # C is {3}
```

Difference: Finding What's Not in the Other Set

The **difference** between Set A and Set B ($A - B$) creates a new set with elements that are in A but *not* in B. **Order is very important!** Use the minus operator `-` or the `.difference()` method.



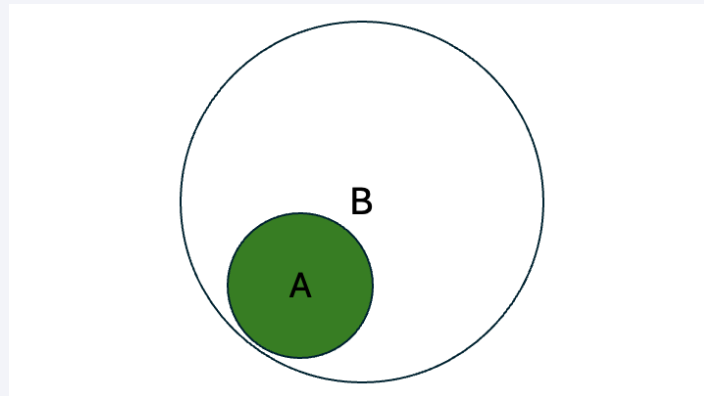
```
A = {1, 2, 3}
B = {3, 4, 5}
C = A - B # C is {1, 2}
D = B - A # D is {4, 5}
```

4. Subset and Superset Relationships

We can also check the relationship between sets.

- A **subset** means that all elements of one set are contained within another set.
- A **superset** is the reverse; it is a set that contains all the elements of another set.

To check for this in Python, use the operators `<=` (subset) and `>=` (superset). If you want to check if it's a *proper* subset (meaning the sets are not equal), use `<` and `>`.



```
A = {1, 2}
B = {1, 2, 3, 4}
print(A <= B) # True
print(A < B)  # True

C = {1, 2}
print(A <= C) # True
print(A < C)  # False, because they are equal
```

5. Knowledge Check Quiz (Python)

1

What are the two fundamental characteristics of a set?

- A. ☐ Unique and Indexed
- B. ☒ Unordered and Unique
- C. ☐ Ordered and Indexed

D. ☐ Ordered and Mutable

2

You have a Python list `data = [5, 6, 7, 5]`. What is the result of `set(data)`

A. ☐ (5, 6, 7)

B. ☐ [5, 6, 7, 5]

C. ☐ [5, 6, 7]

D. ☒ {5, 6, 7}

3

Which Python method should you use to remove an element from a set without causing an error if the element doesn't exist?

A. ☐ `.remove()`

B. ☒ `.discard()`

C. ☐ `.delete()`

D. ☐ `.pop()`

4

Given $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$, what is $A \cup B$ (Union)?

A. ☒ {1, 2, 3, 4, 5}

B. ☐ {3}

C. ☐ {1, 2, 3, 3, 4, 5}

D. ☐ {1, 2, 4, 5}

5

Given $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$, what is $A \cap B$ (Intersection)?

A. ☐ {4, 5}

B. ☐ {1, 2, 3, 4, 5}

C. ☒ {3}

D. ☐ {1, 2}

6

Which Python operator is used to find the difference between setX and setY?

A. ☒ -

B. ☐ ^

C. ☐ &

D. ☐ |

7

If $X = \{a, b\}$ and $Y = \{a, b, c\}$, which statement is TRUE?

A. ☐ $X == Y$

B. ☒ $X \leq Y$

C. ☐ The union of X and Y is {c}.

D. ☐ $Y \leq X$

8

How do you create an empty set in Python?

A. ☐ []

B. ☐ ()

C. ☐ {}

D. ☒ set()

9

What will `my_set.remove('z')` do if 'z' is not in `my_set`?

A. ☐ Return False

B. ☒ Raise a KeyError

C. ☐ Return None

D. ☐ Do nothing

10

What is a primary real-world use case for a set?

A. ☒ Keeping track of all unique tags on a blog post.

B. ☐ Storing the pixels of an image in the correct order.

C. ☐ Storing a sequence of user actions in the order they occurred.

D. ☐ Creating a mapping from a student ID to their grade.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**